



Phân loại ảnh sử dụng Mạng neural tích chập (Convolutional neural networks) - CNN

Dẫn nhập

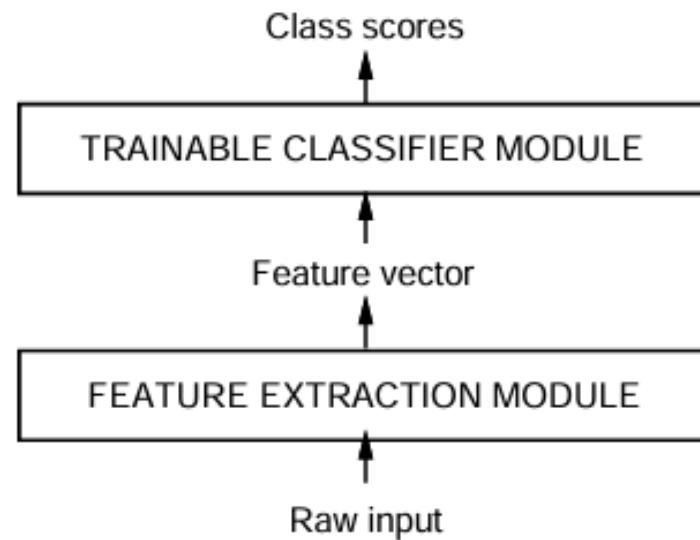
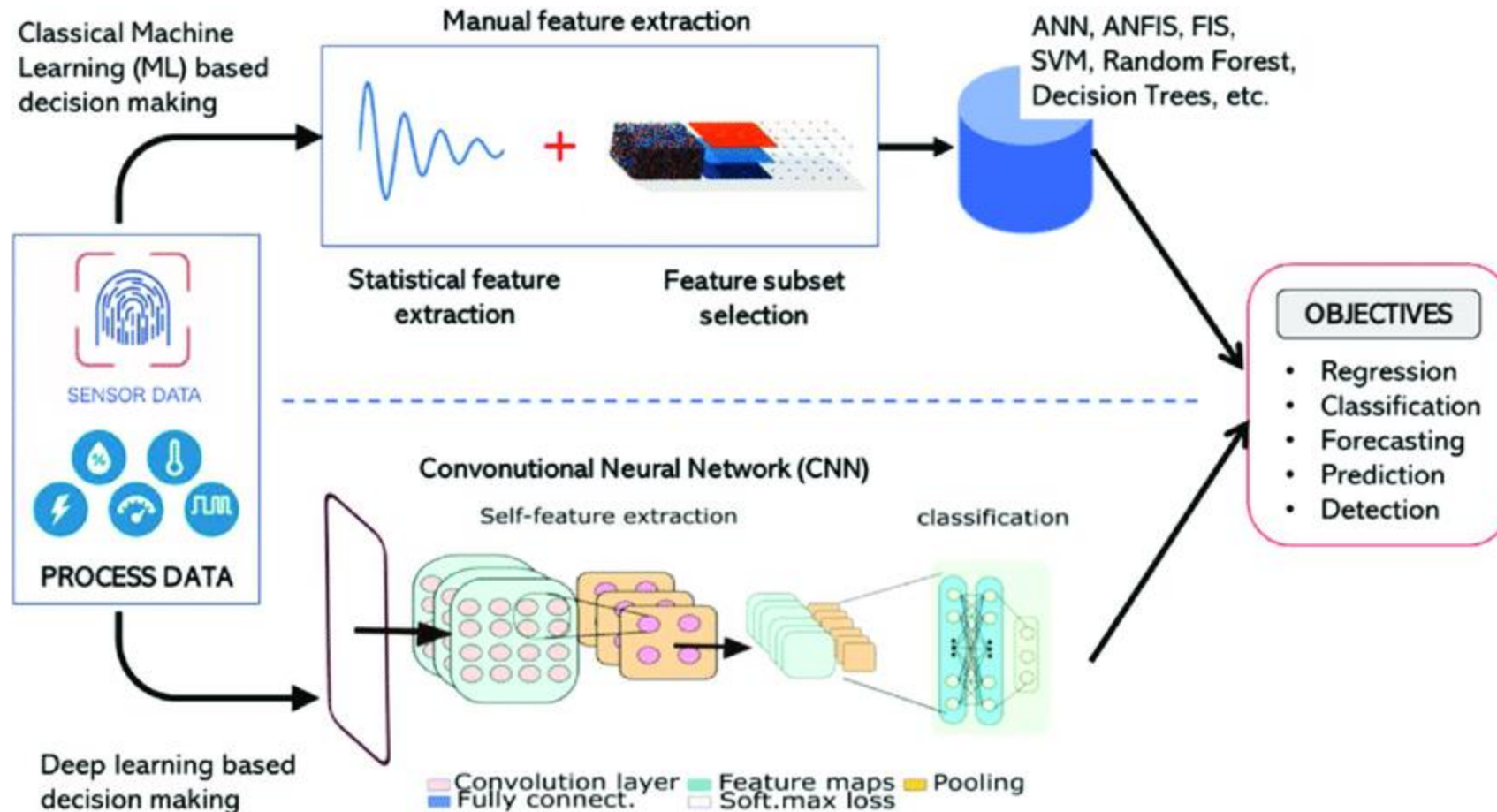


Fig. 1. Traditional pattern recognition is performed with two modules: a fixed feature extractor, and a trainable classifier.

http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf

Dẫn nhập



<https://linkinghub.elsevier.com/retrieve/pii/S1526612520303923>



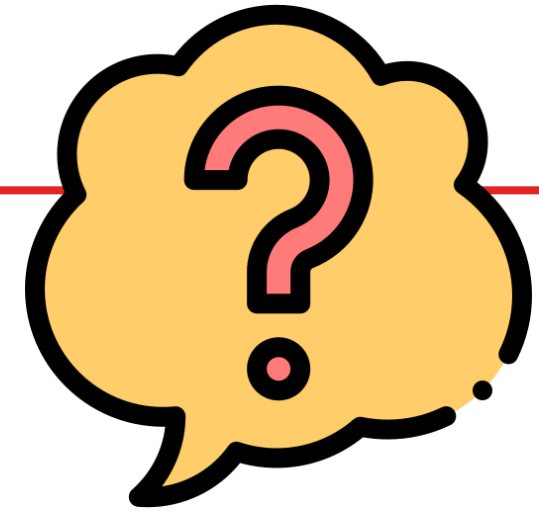
Mục tiêu bài học

- Hiểu được kiến trúc cơ bản của CNN và vai trò của từng thành phần.
- Biết cách xây dựng mô hình CNN đơn giản để phân loại ảnh.
- Đánh giá được hiệu quả mô hình.



Tổng quan về mạng neural tích chập

1. Mạng CNN là gì ?



Mạng neural tích chập (Convolutional neural networks) - CNN

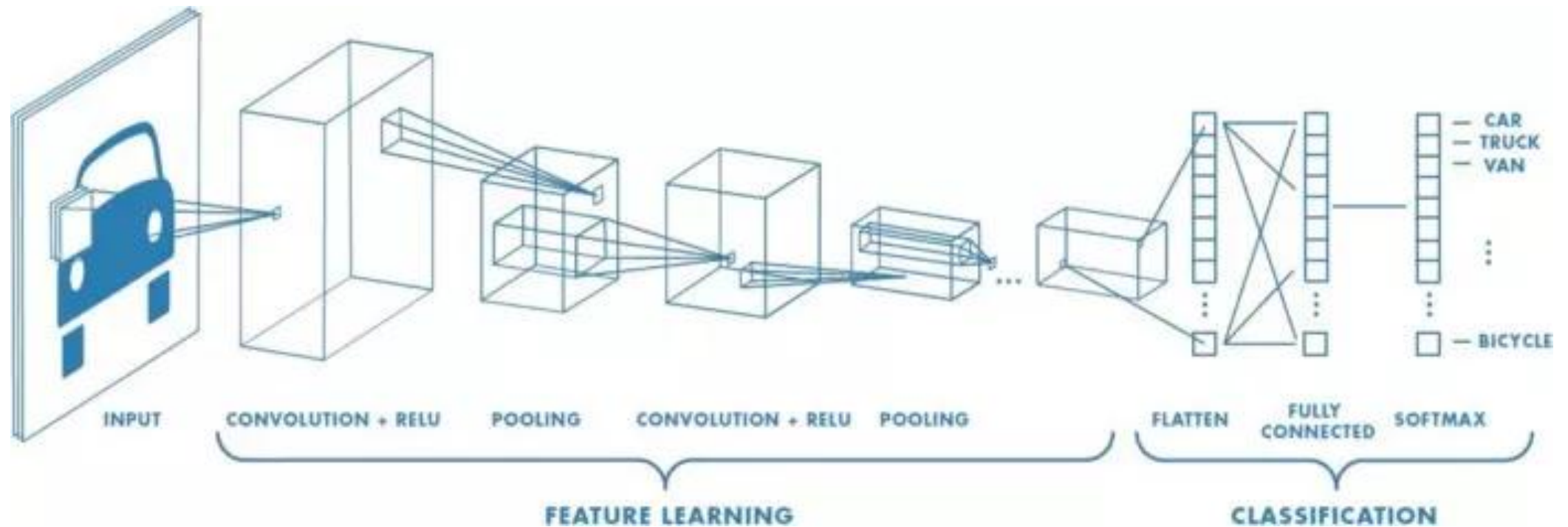
 Copilot Answer

Mạng nơ-ron tích chập (CNN) là một loại mô hình học sâu được thiết kế đặc biệt để phân tích dữ liệu trực quan như hình ảnh và video. **CNN tự động học các đặc trưng không gian từ dữ liệu đầu vào**, giúp nhận dạng và phân loại hình ảnh, video, và ngôn ngữ tự nhiên. Mạng này được sử dụng rộng rãi trong lĩnh vực **thị giác máy tính** và xử lý hình ảnh, với khả năng xây dựng các hệ thống có độ chính xác cao trong nhận diện vật thể và khuôn mặt.

⇌ Ultralytics +4

1. Mạng CNN là gì ?

Ví dụ:



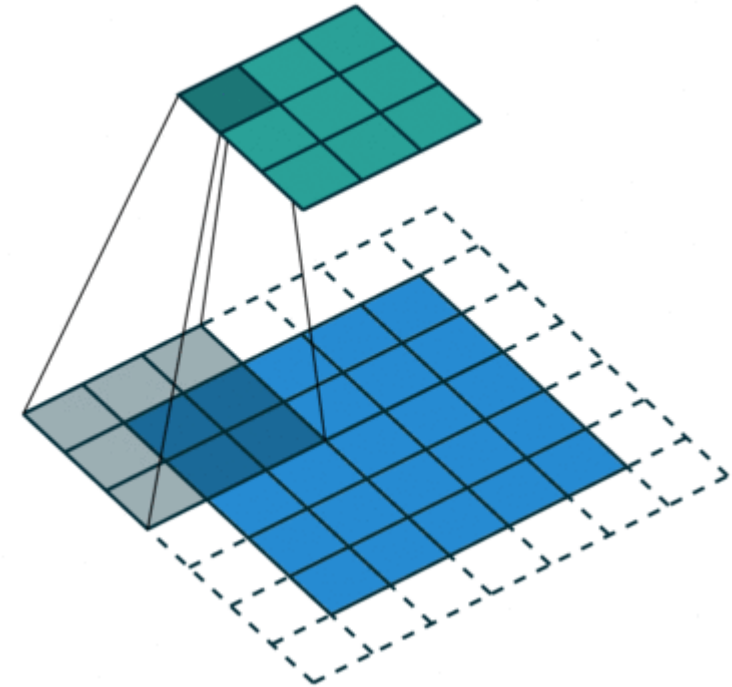


2. Các thành phần cơ bản

- **Convolutional Layer**
- **Activation Layer / Function**
- **Pooling Layer**

- Flatten Layer
- Fully Connected Layer
- Softmax

Convolutional layer





Convolutional Layer

Chức năng:

- **Trích xuất đặc trưng** từ ảnh đầu vào bằng cách thực hiện phép nhân tích chập (convolution) với các bộ lọc.
- **Tạo ra một bản đồ đặc trưng** (feature map) hay activation map thể hiện đặc điểm quan trọng trong ảnh.

Convolutional Layer

Phép tính tích chập:

2	4	9	1	4
2	1	4	4	6
1	1	2	9	2
7	3	5	1	3
2	3	4	8	5

×

1	2	3
-4	7	4
2	-5	1

=

?

Convolutional Layer

Phép tính tích chập:

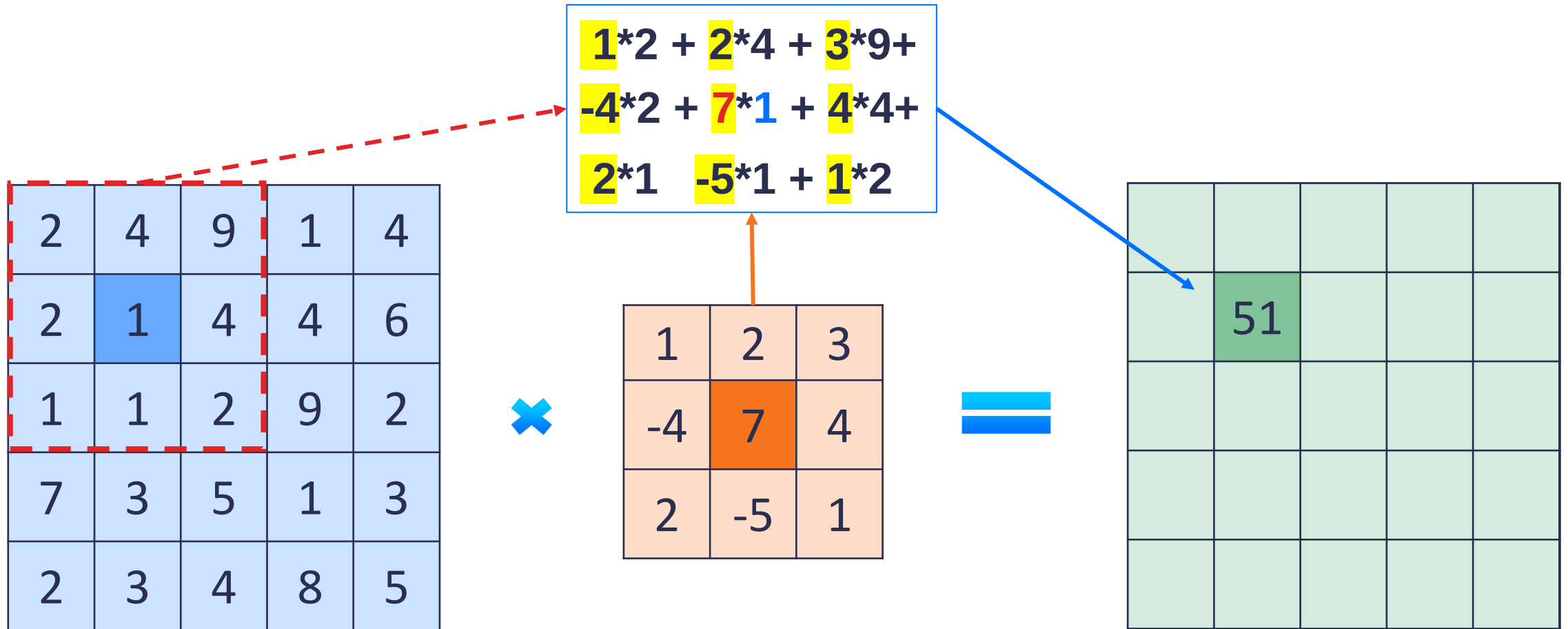
2	4	9	1	4
2	1	4	4	6
1	1	2	9	2
7	3	5	1	3
2	3	4	8	5

×

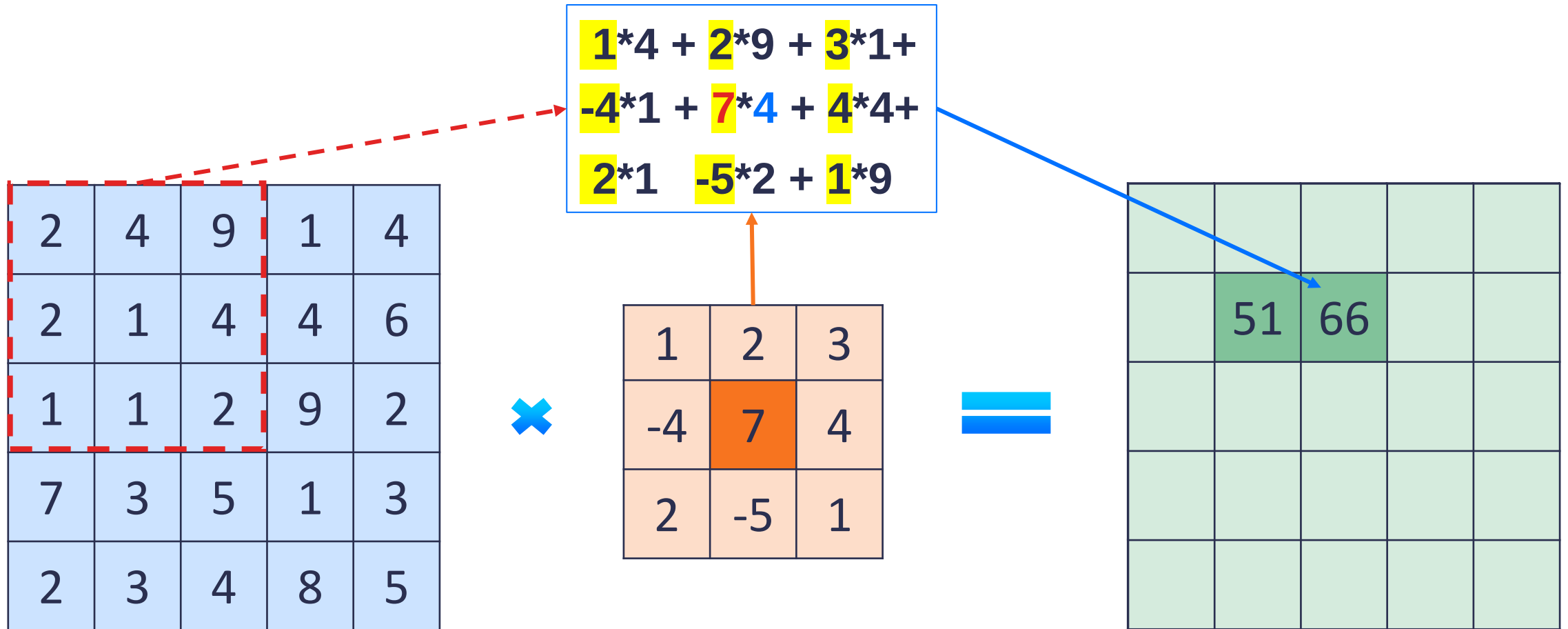
1	2	3
-4	7	4
2	-5	1

=

Convolutional Layer



Convolutional Layer





Convolutional Layer

Stride
→

S=1

2	4	9	1	4
2	1	4	4	6
1	1	2	9	2
7	3	5	1	3
2	3	4	8	5

×

1	2	3
-4	7	4
2	-5	1

=

	51	66		



Convolutional Layer

Stride

- Bước di chuyển của kernel khi quét qua ảnh (stride = 1: di chuyển từng pixel) .

Nguồn: <https://stanford.edu/~shervine/l/vi/teaching/cs-230/cheatsheet-convolutional-neural-networks>

Convolutional Layer



2	4	9	1	4
2	1	4	4	6
1	1	2	9	2
7	3	5	1	3
2	3	4	8	5



1	2	3
-4	7	4
2	-5	1

Padding

$$\begin{aligned} &1*1 + 2*4 + 3*? + \\ &-4*4 + 7*6 + 4*? + \\ &2*9 - 5*2 + 1*? \end{aligned}$$

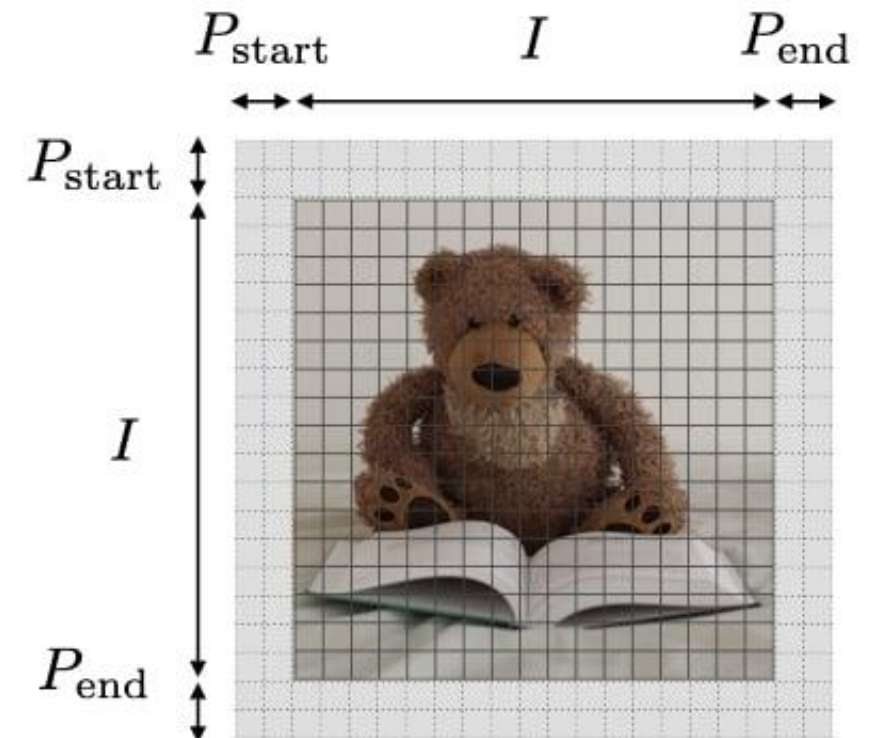


	51	66	20	

Convolutional Layer

Padding

- Là những điểm ảnh được thêm vào để tính được giá trị tích chập cho các điểm ảnh ở vị trí biên $\rightarrow$ hạn chế mất thông tin.



Convolutional Layer

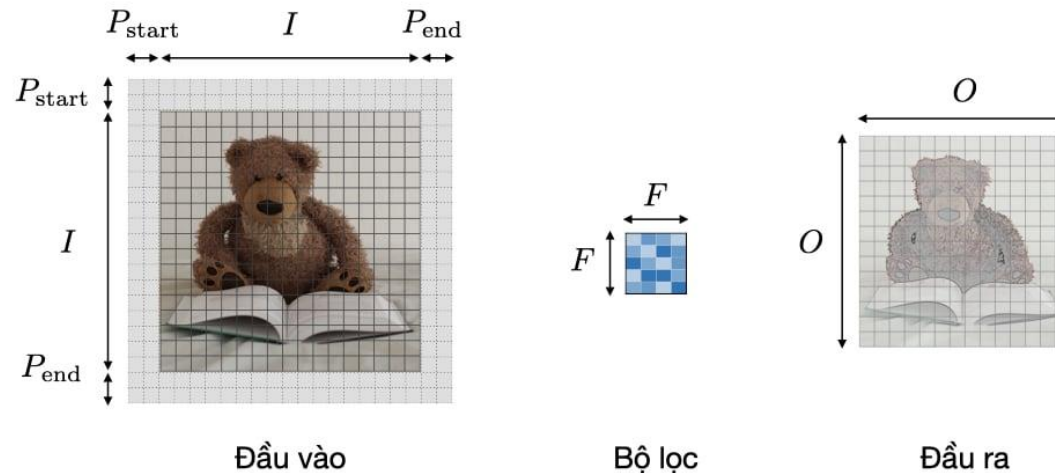
Padding

Phương pháp	Valid	Same	Full
Giá trị	$P = 0$	$P_{start} = \left\lfloor \frac{S \left\lceil \frac{I}{S} \right\rceil - I + F - S}{2} \right\rfloor$ $P_{end} = \left\lceil \frac{S \left\lceil \frac{I}{S} \right\rceil - I + F - S}{2} \right\rceil$	$P_{start} = \llbracket 0, F - 1 \rrbracket$ $P_{end} = F - 1$
Mục đích	• Không sử dụng padding • Bỏ phép tích chập cuối nếu số chiều không khớp	• Sử dụng padding để làm cho feature map có kích thước $\lceil I/S \rceil$ • Còn được gọi là 'half' padding	• Padding tối đa sao cho các phép tích chập có thể được sử dụng tại các rìa của đầu vào

Convolutional Layer

Tính toán kích thước đầu ra

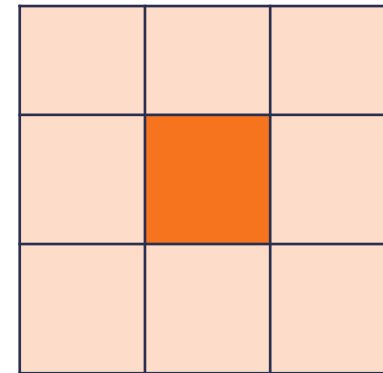
- Ký hiệu I là độ dài kích thước đầu vào, F là độ dài của bộ lọc, P là số lượng zero padding, S là độ trượt, ta có thể tính được độ dài O của feature map theo một chiều bằng công thức:
$$O = \frac{I - F + P_{start} + P_{end}}{S} + 1$$



Nguồn: <https://stanford.edu/~shervine/l/vi/teaching/cs-230/cheatsheet-convolutional-neural-networks>

Convolutional Layer

Giá trị của các bộ lọc được xác định bằng cách nào ?



→ Được “học” từ dữ liệu.

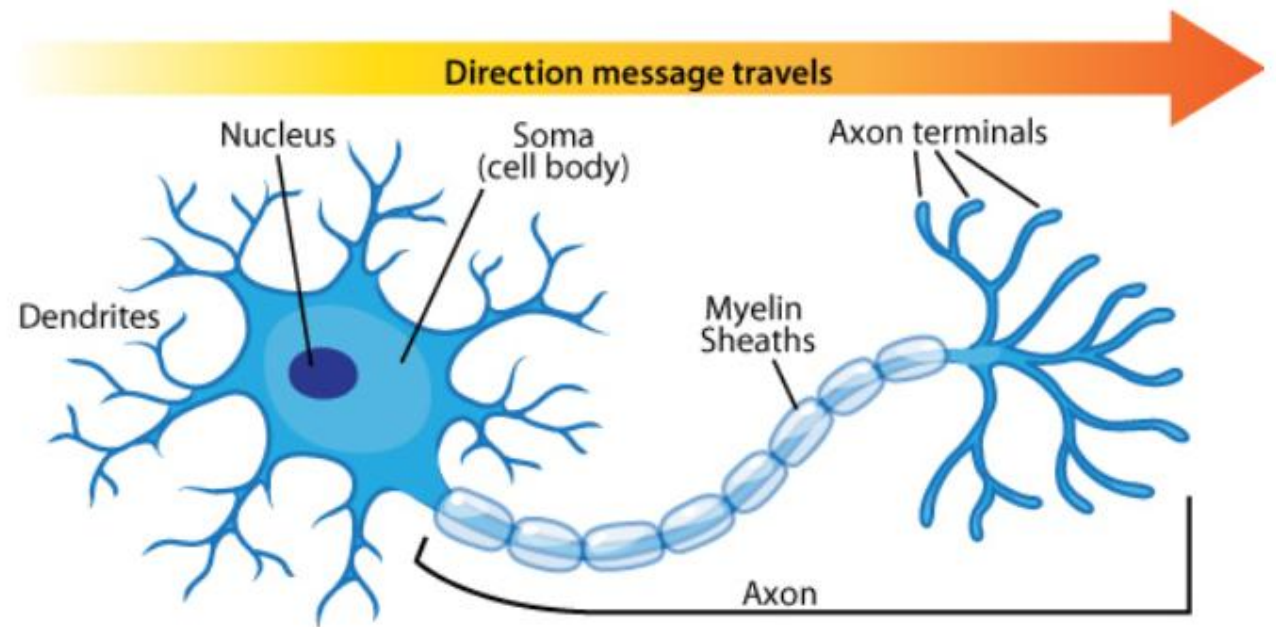
→ Sử dụng phương pháp học của mạng neural nhân tạo.

Convolutional Layer

Activation Layer / Neuron Anatomy

Function:

→ quyết định liệu một neuron có được kích hoạt (active) hay không ?



Nguồn: <https://askabiologist.asu.edu/neuron-anatomy>



Convolutional Layer

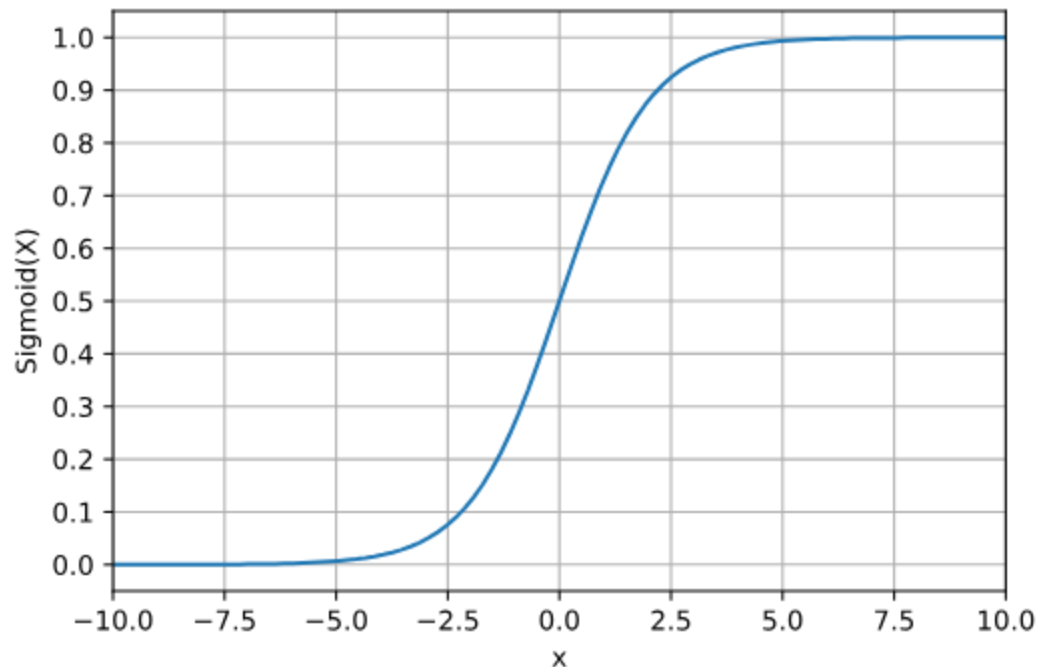
Activation Function:

- Có chức năng quyết định liệu một neuron có được kích hoạt (**active**) hay không ?
- Là một hàm kích hoạt g được sử dụng nhằm giúp mạng học được **mối quan hệ phi tuyến** của các giá trị đầu ra của một neuron.
- Một số hàm phổ biến: ReLU, Sigmoid, Tanh

Activation Function

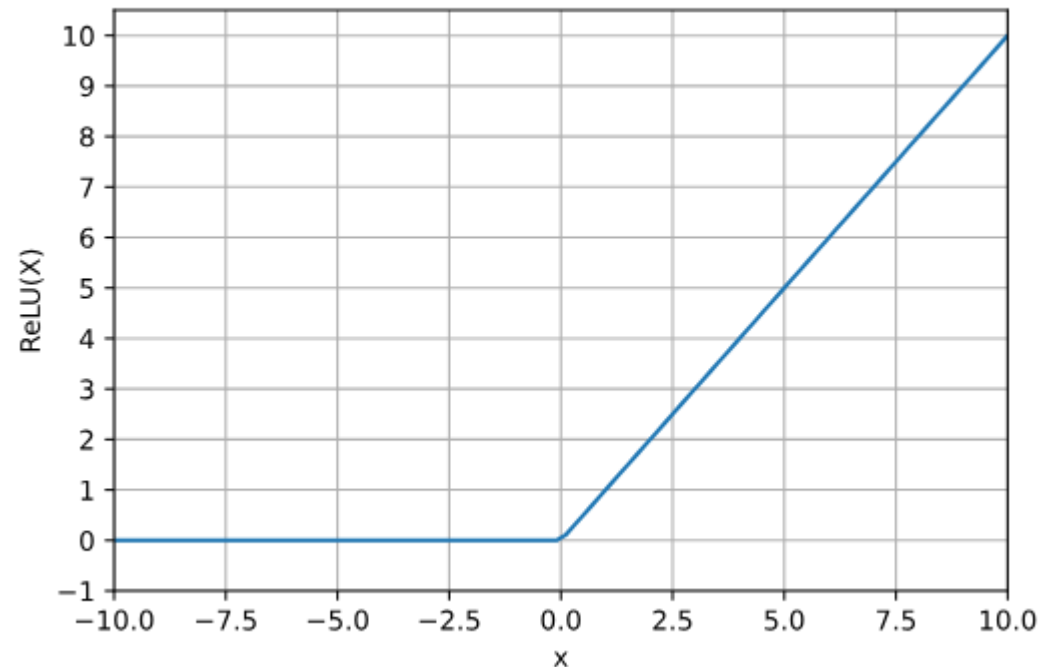
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



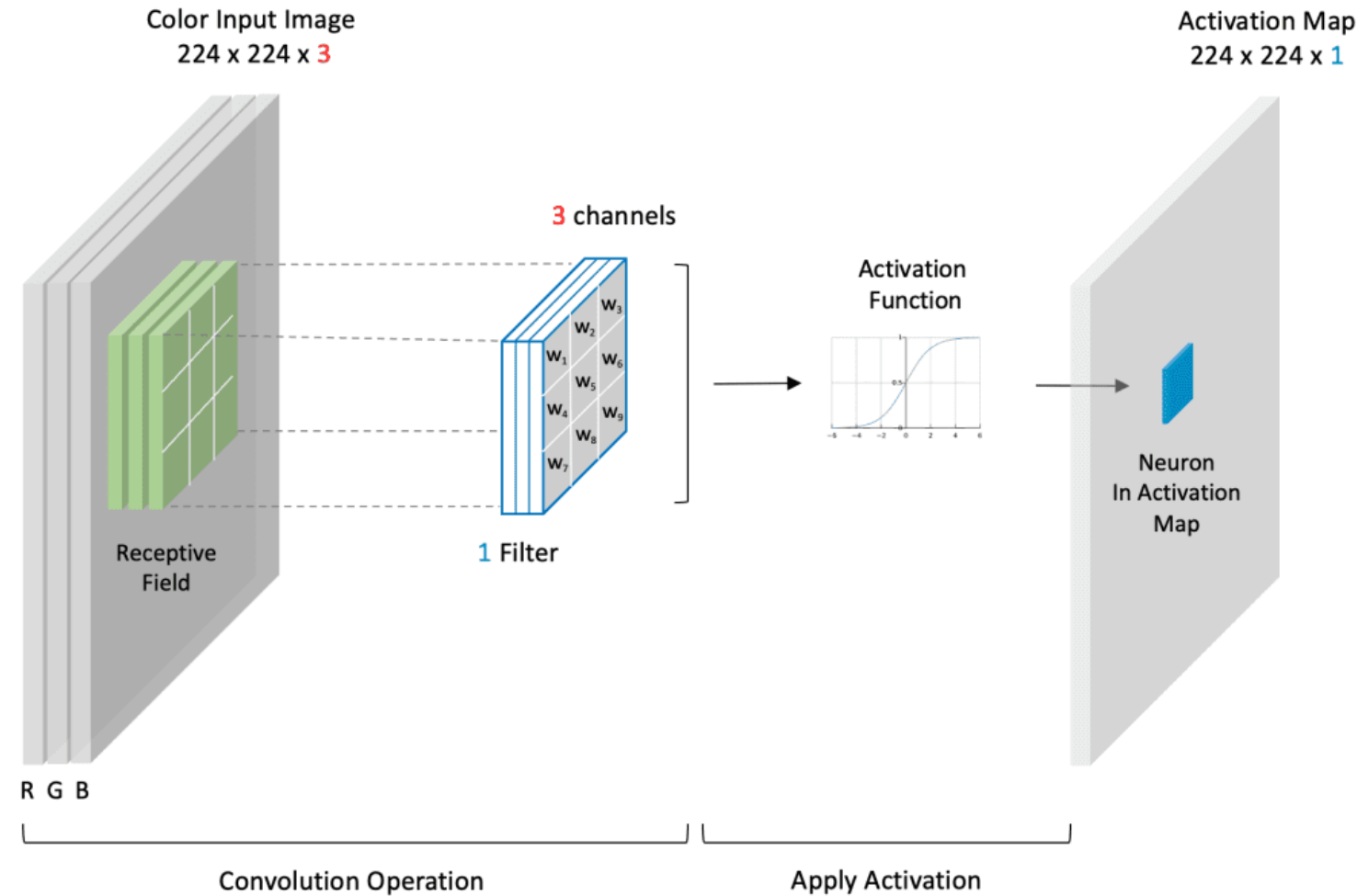
ReLU

$$f(x) = \max(0, x)$$

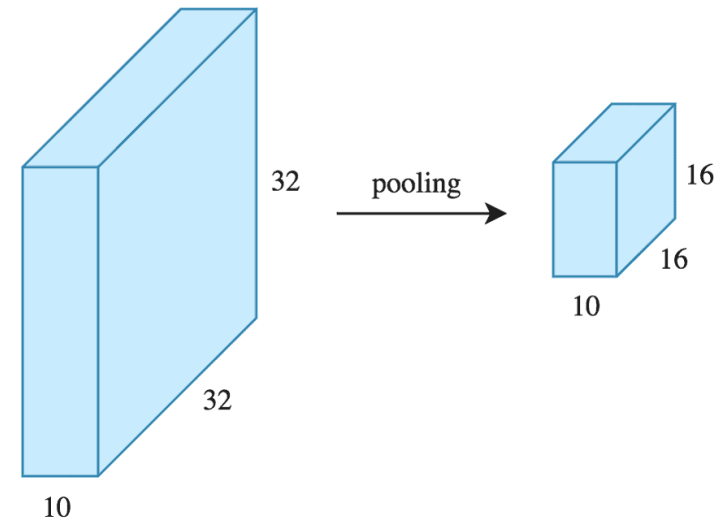
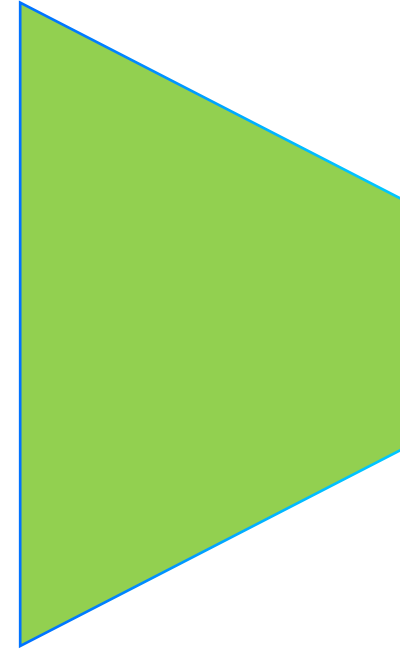


Convolutional Layer

- Ví dụ



Pooling Layer



Pooling Layer

- Tầng pooling (POOL) là một phép **downsampling**, thường được sử dụng sau tầng tích chập.
- Chức năng
 - Giúp tăng tính bất biến không gian.
 - **Giảm kích thước của feature map.**
- Một số phương pháp: **Max Pooling**, Average Pooling,...

Pooling Layer

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Max Pool
→
Filter - (2 x 2)
Stride - (2, 2)

9	7
8	6

Nguồn: <https://www.geeksforgeeks.org/cnn-introduction-to-pooling-layer/>

Pooling Layer

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

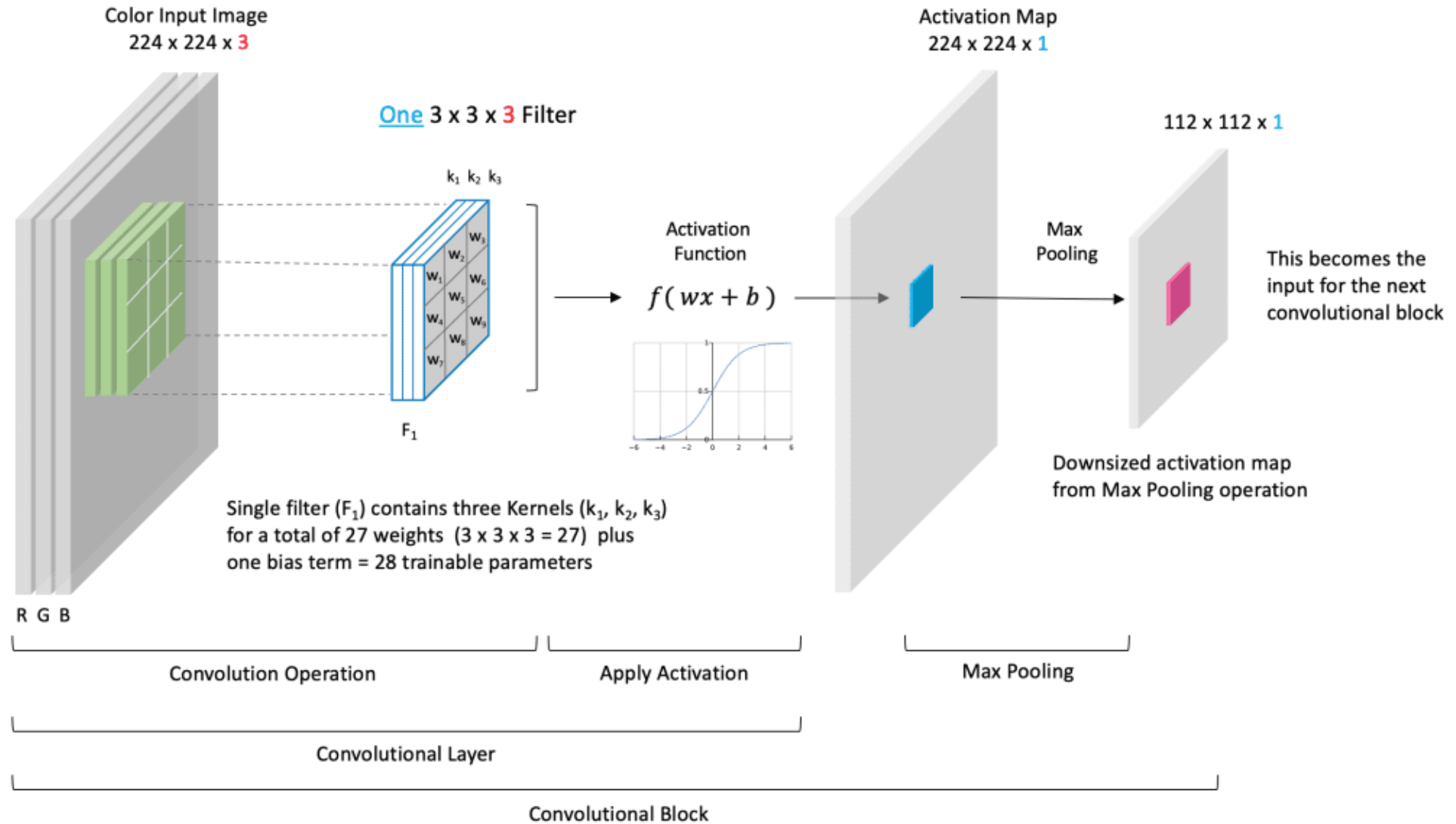
Average Pool
→
Filter - (2 x 2)
Stride - (2, 2)

4.25	4.25
4.25	3.5

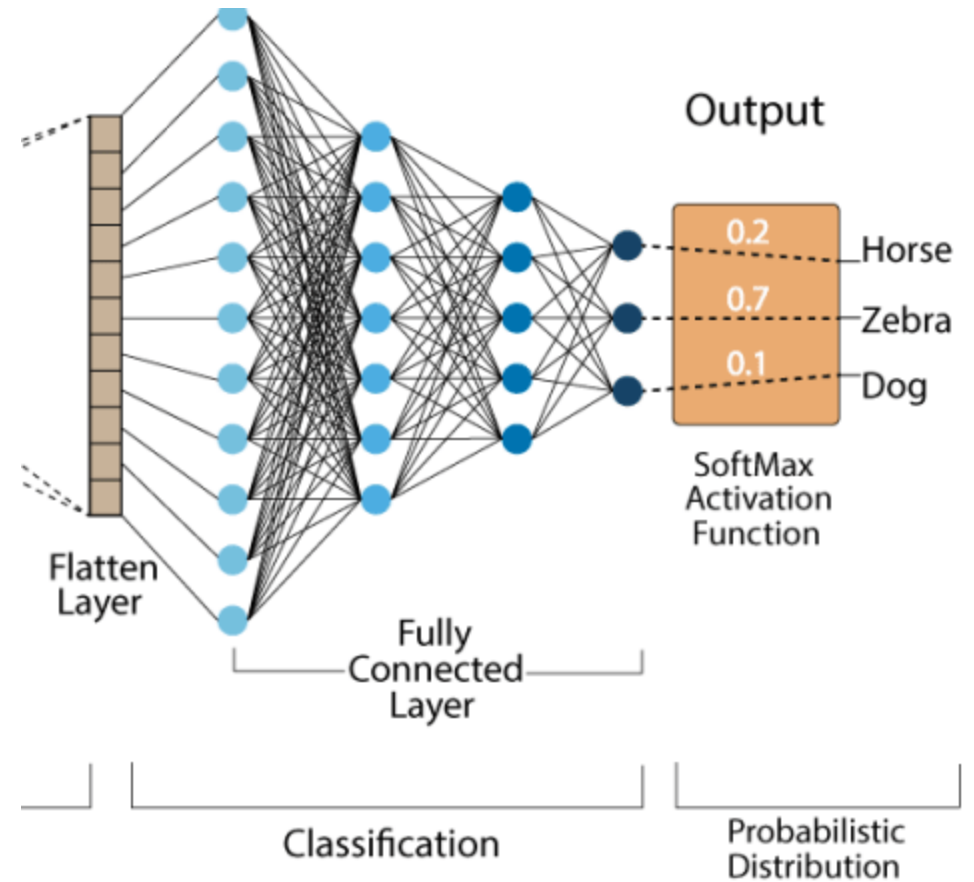
Nguồn: <https://www.geeksforgeeks.org/cnn-introduction-to-pooling-layer/>



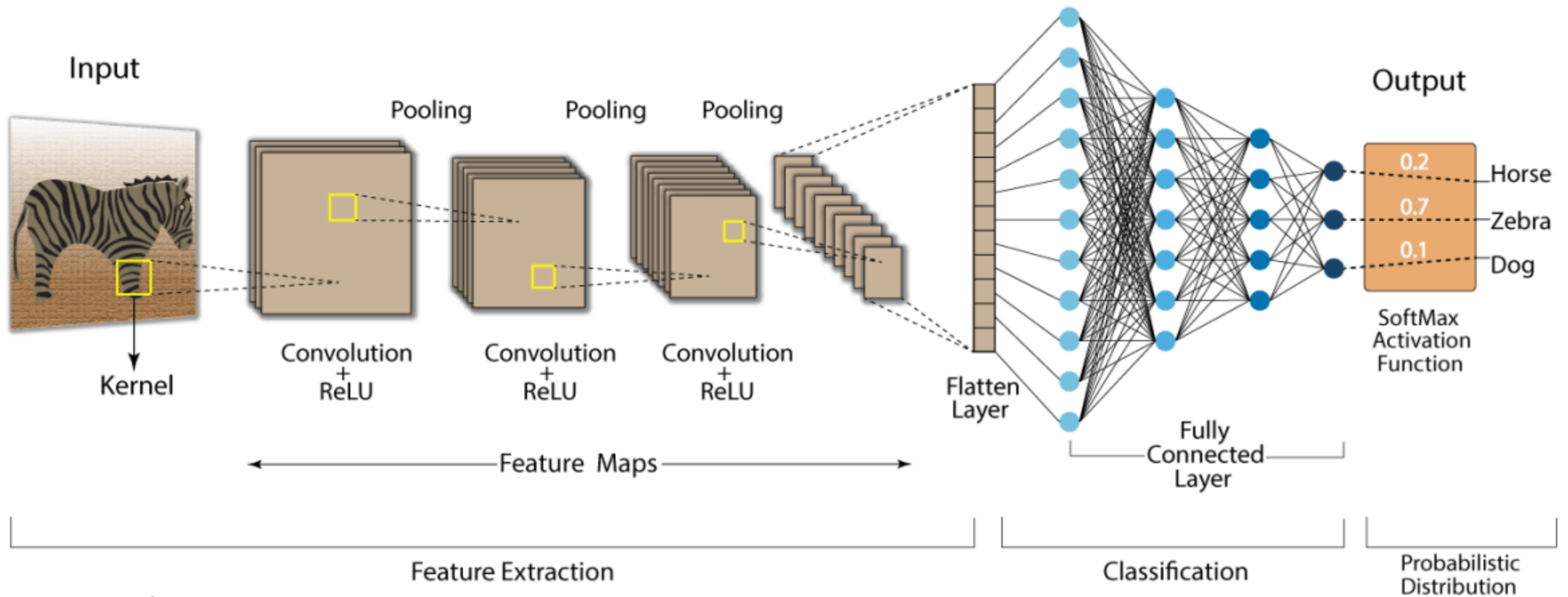
Convolutional Block



Top of the network



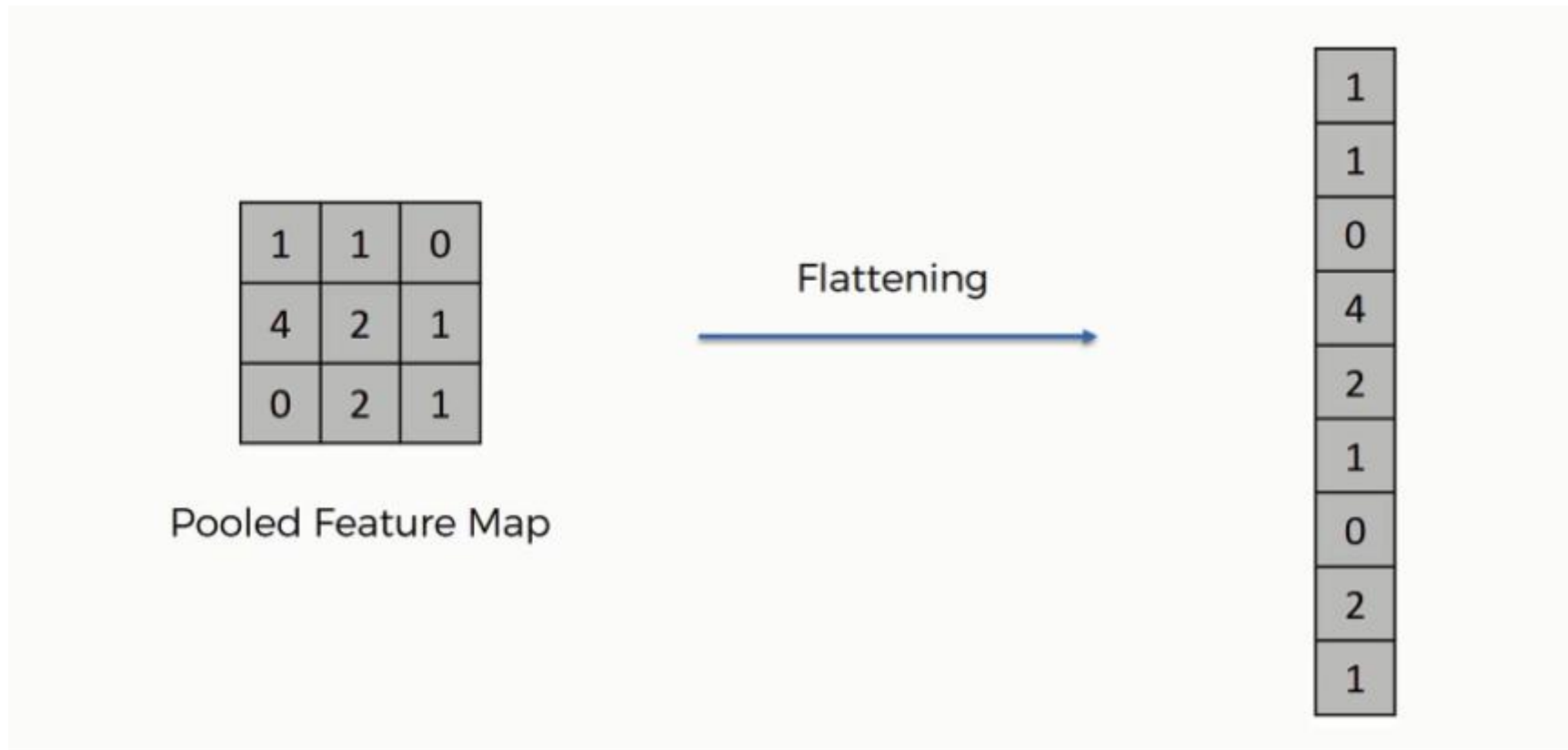
Top of the network



Nguồn: developersbreach.com

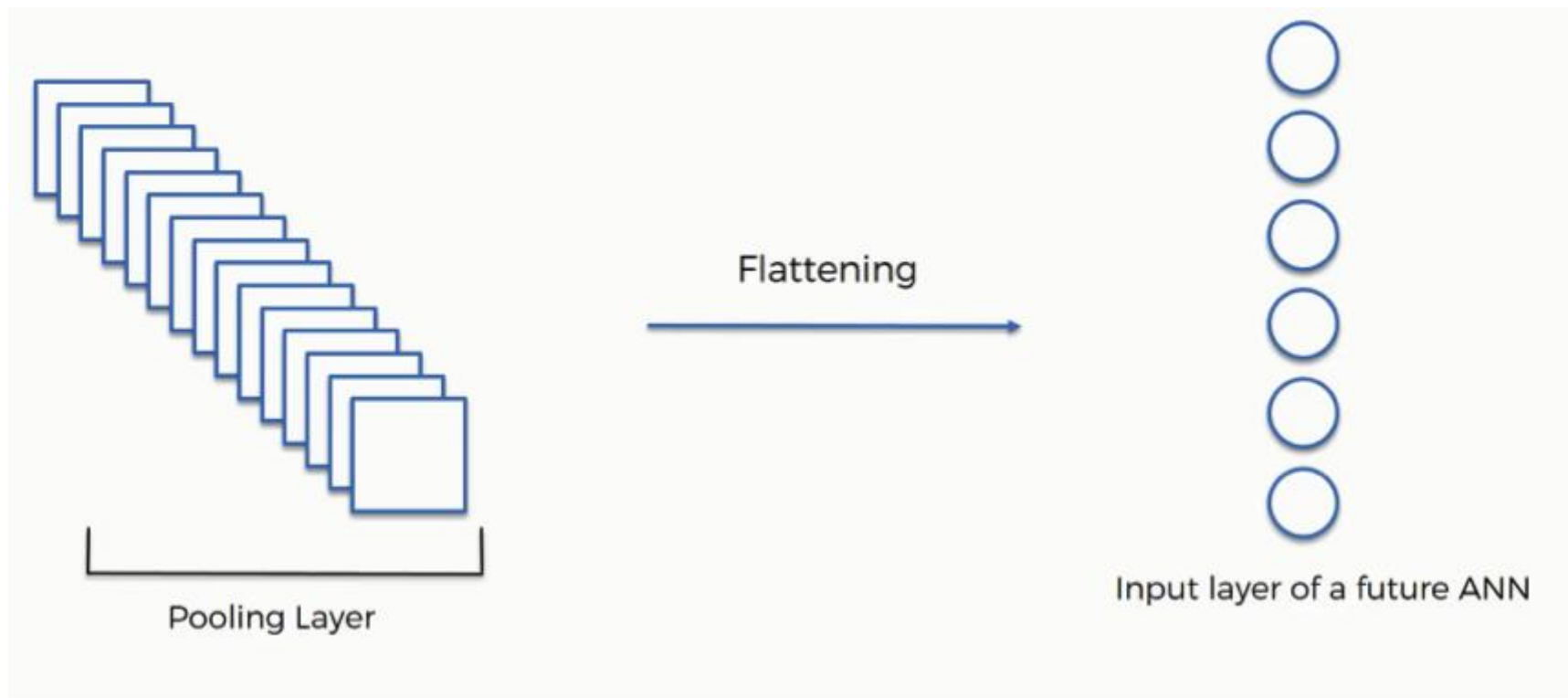
Flattern layer

- Có chức năng duỗi thẳng ma trận thành 1 vector



Flattern layer

- Nếu có nhiều vector thì nối tất cả các vector này thành 1 vector duy nhất → tạo thành vector đặc trưng

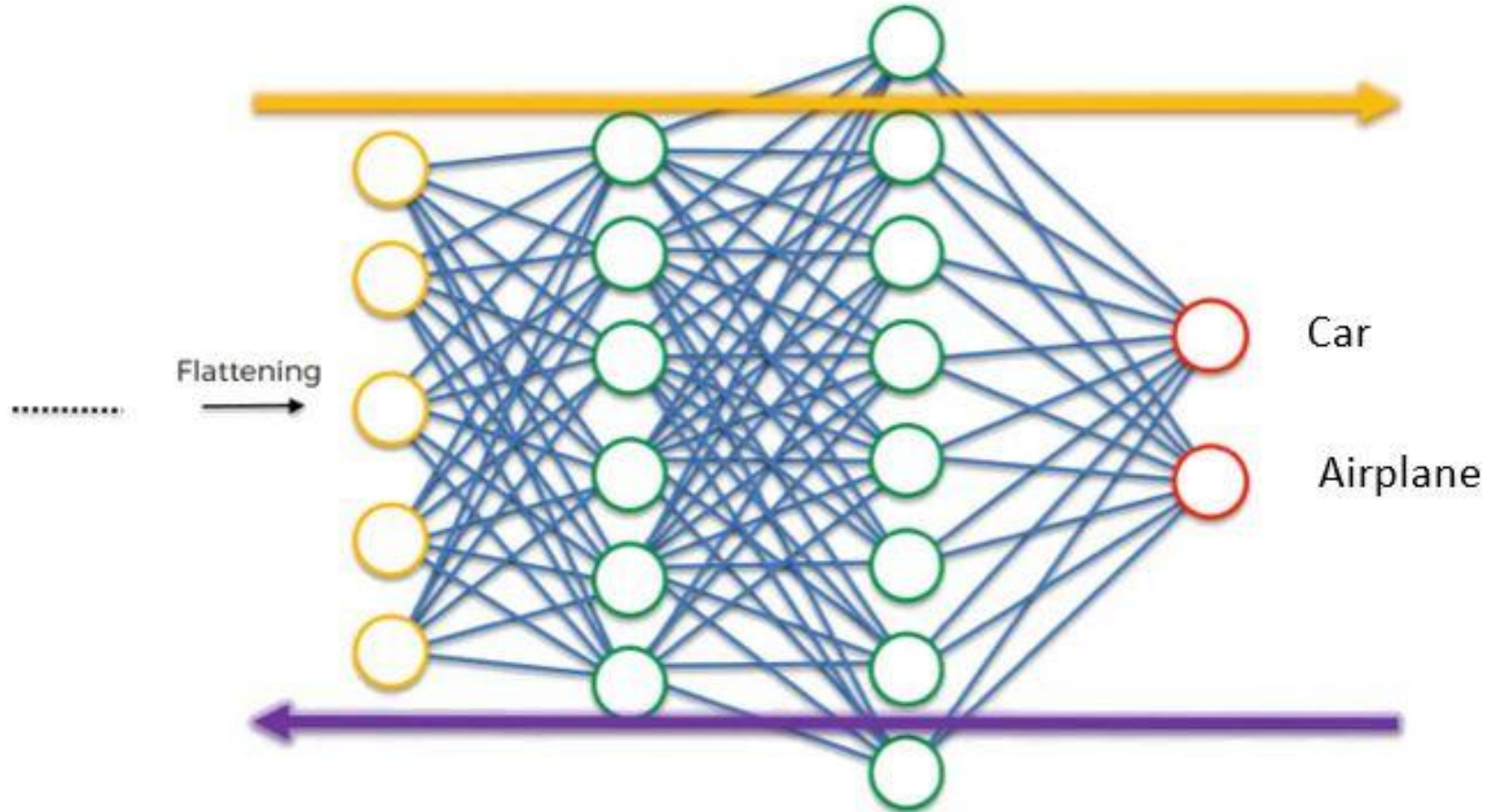




Fully Connected Layer

- Nhận **đầu vào** là các dữ liệu đã được **làm phẳng**, mà mỗi đầu vào đó được kết nối đến tất cả neuron.
- Các tầng kết nối đầy đủ thường được tìm thấy **ở cuối mạng** và được dùng để tối ưu hóa **mục tiêu của mạng** ví dụ như độ chính xác của lớp.
- Kích thước **đầu ra** phụ thuộc vào **số lượng lớp** đối tượng / số lượng giá trị kết quả.

Fully Connected Layer



Softmax Layer

- Softmax có thể được coi là một hàm logistic tổng quát lấy đầu vào là một vector chứa các giá trị $x \in \mathbb{R}^n$ và cho ra là một vector gồm các xác suất $p \in \mathbb{R}^n$ thông qua một hàm softmax ở cuối kiến trúc.

- Định nghĩa như sau:

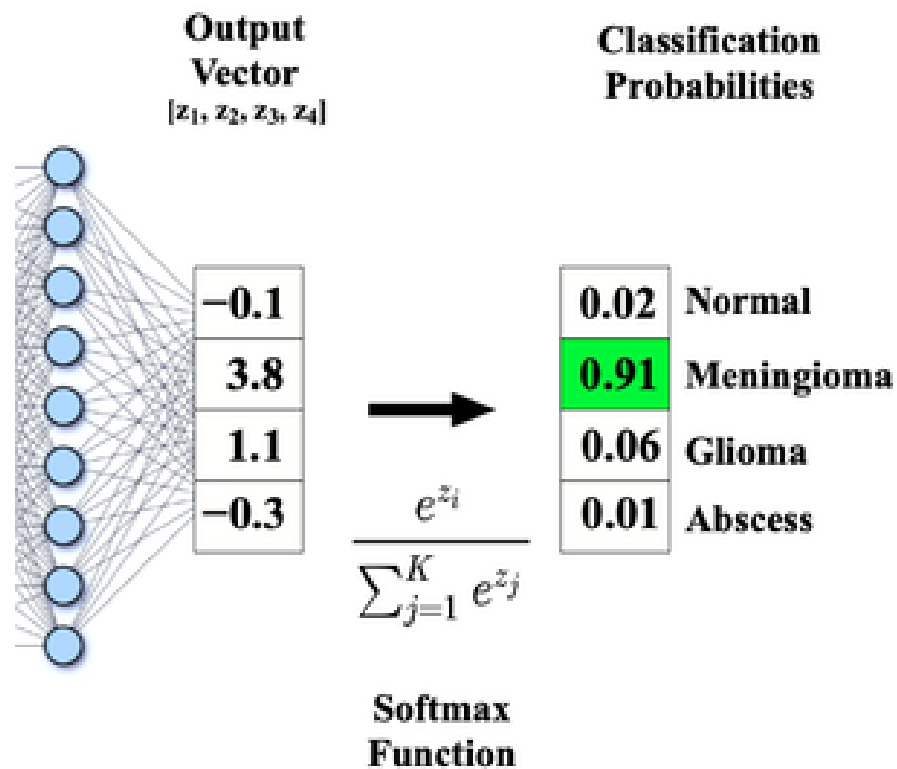
$$p = \begin{pmatrix} p_1 \\ \vdots \\ p_n \end{pmatrix}$$

với

$$p_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Softmax Layer

- Ví dụ



$$\text{Softmax}(z_2) = \text{Softmax}(3.8) = \frac{e^{3.8}}{e^{-0.1} + e^{3.8} + e^{1.1} + e^{-0.3}} = 0.91$$



Ước lượng Độ phức tạp của mô hình

Ước lượng Độ phức tạp của mô hình

- Dựa trên **số lượng tham số** cần phải “học” trong mô hình

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 6)	156
max_pooling2d (MaxPooling2D)	(None, 14, 14, 6)	0
conv2d_1 (Conv2D)	(None, 10, 10, 16)	2,416
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 16)	0
flatten (Flatten)	(None, 400)	0
dense (Dense)	(None, 120)	48,120
dense_1 (Dense)	(None, 84)	10,164
dense_2 (Dense)	(None, 10)	850

Total params: 61,706 (241.04 KB)
Trainable params: 61,706 (241.04 KB)
Non-trainable params: 0 (0.00 B)



Convolutional Layer

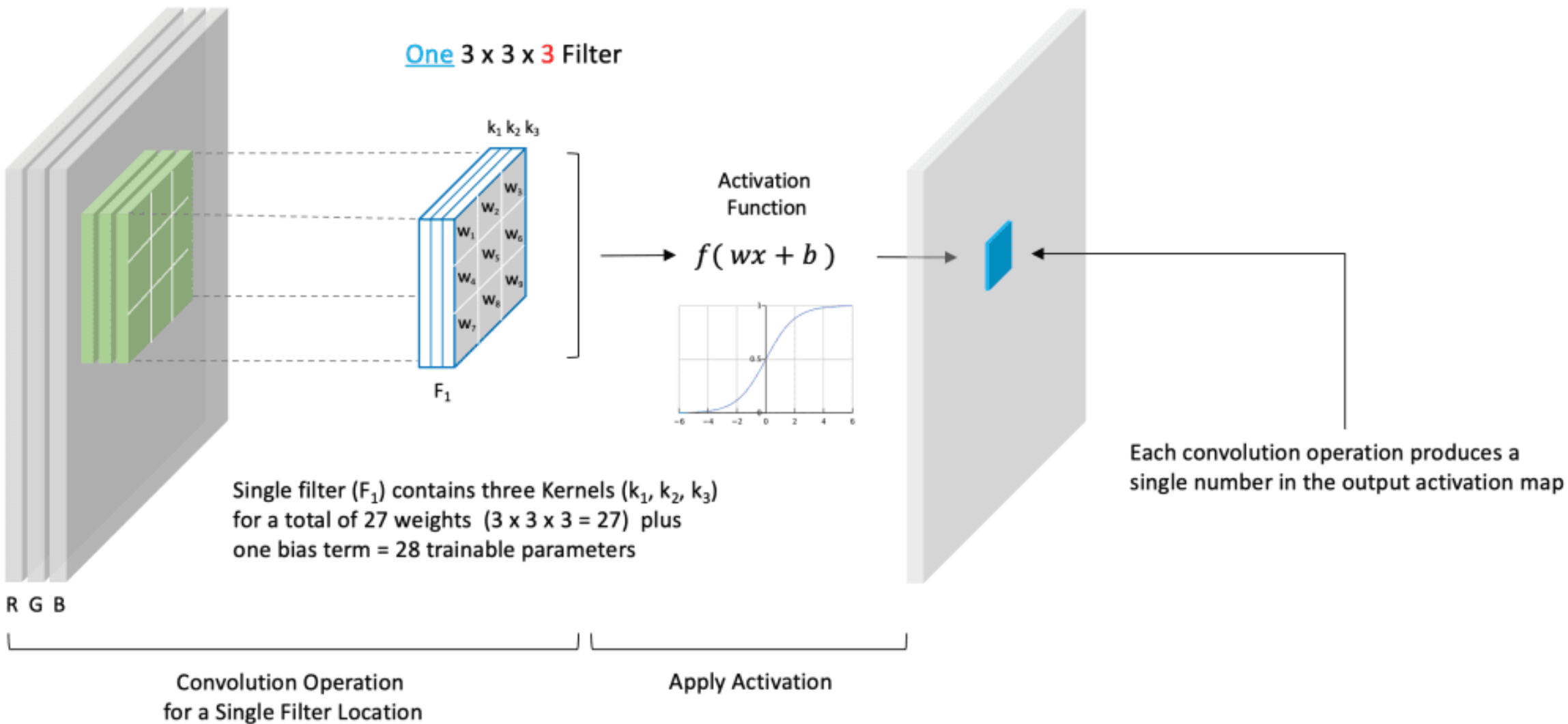
Kích thước bộ lọc:

- Ảnh đầu vào kích thước $I \times I \times C$ chứa C kênh (channels) thì bộ lọc có kích thước $F \times F \times C$
- Feature map kết quả có kích thước $O \times O \times 1$.

Color Input Image
224 x 224 x 3

Activation Map
224 x 224 x 1

One 3 x 3 x 3 Filter



Nguồn: <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1



308

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2



-498

0	1	1
0	1	0
1	-1	1

Kernel Channel #3



164

+

+

+ 1 = -25

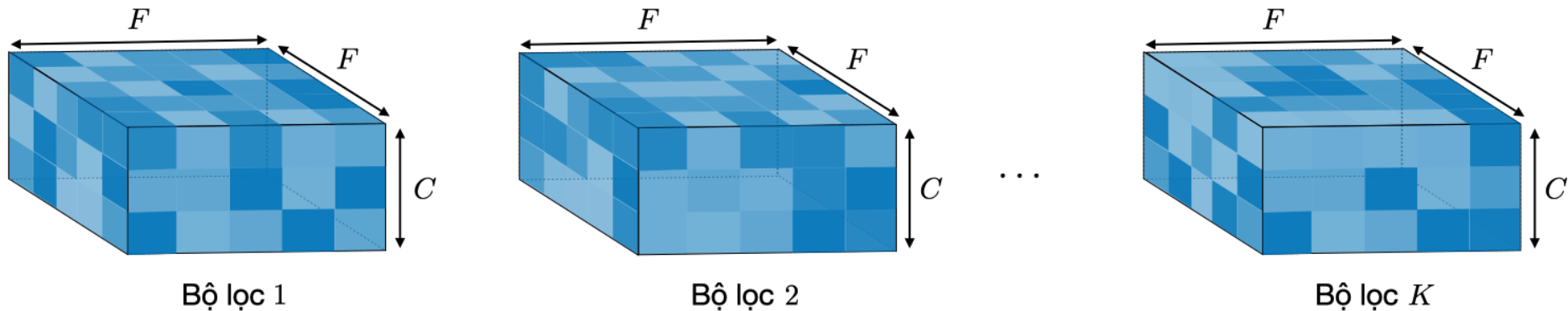
Bias = 1

Output

-25				...
				...
				...
				...
...	...	...	...	...

Convolutional Layer

- Để trích chọn được nhiều thông tin, ta thường sử dụng nhiều hơn 1 bộ lọc



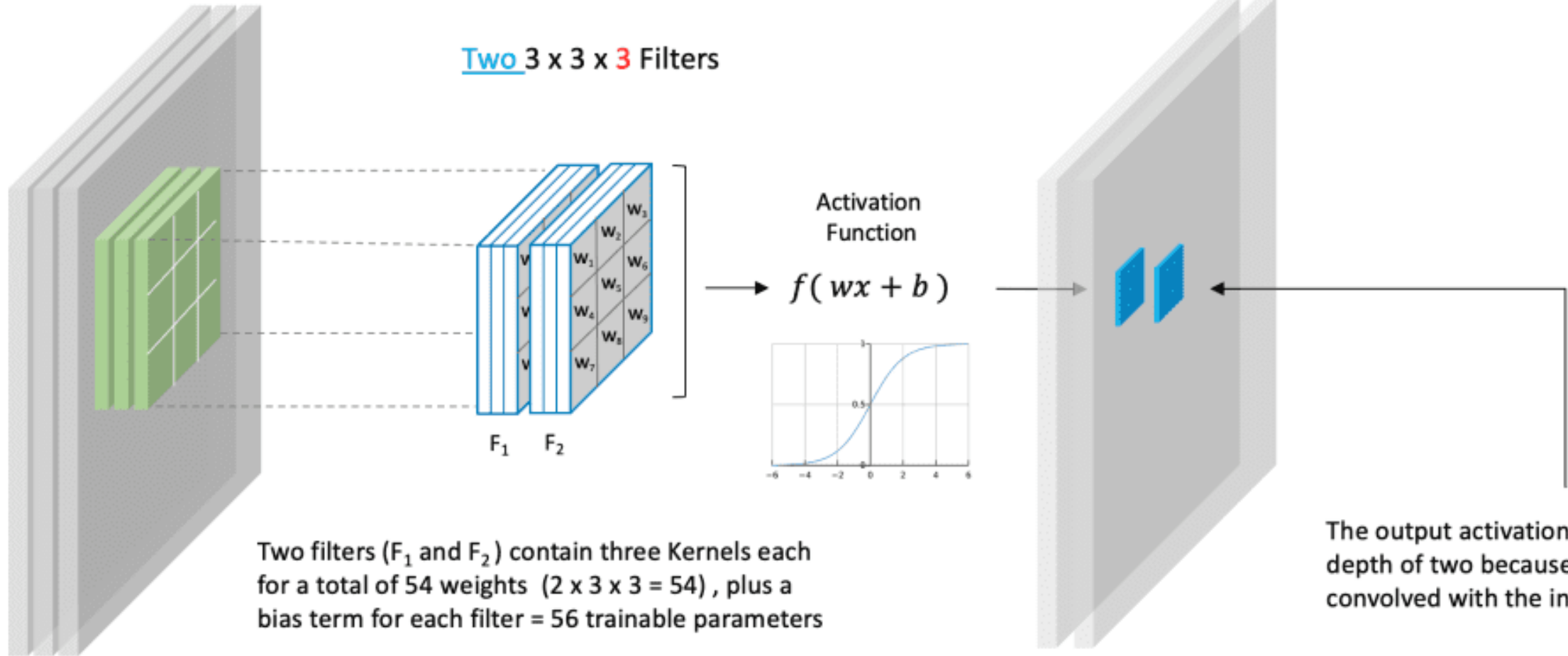
Việc áp dụng K bộ lọc có kích thước $F \times F$ cho ra một feature map có kích thước $O \times O \times K$.

Nguồn: <https://stanford.edu/~shervine/l/vi/teaching/cs-230/cheatsheet-convolutional-neural-networks>

Input Tensor
 $w \times h \times 3$

Activation Map
 $w \times h \times 2$

Two $3 \times 3 \times 3$ Filters

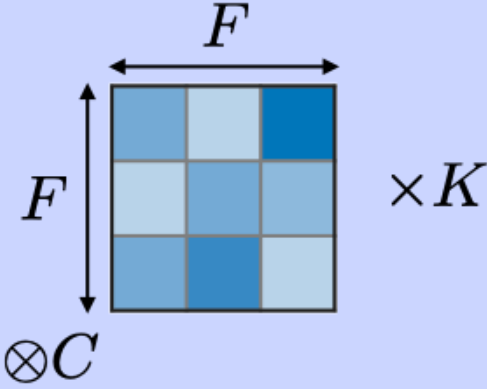
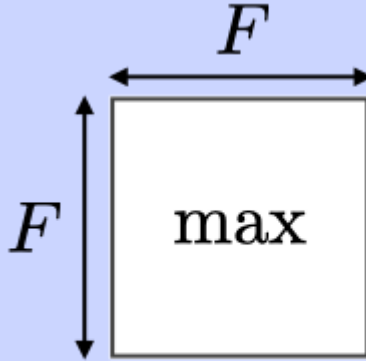
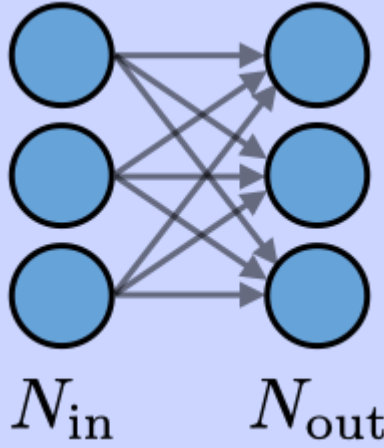


Convolution Operation
for a Single Filter Location

Apply Activation

Nguồn: <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>

Xác định số lượng tham số

	CONV	POOL	FC
Minh họa			
Kích thước đầu vào	$I \times I \times C$	$I \times I \times C$	N_{in}
Kích thước đầu ra	$O \times O \times K$	$O \times O \times C$	N_{out}
Số lượng tham số	$(F \times F \times C + 1)K$	0	$(N_{in} + 1) \times N_{out}$



Xác định số lượng tham số

```
# Build LeNet-5 model
model = models.Sequential()
model.add(layers.Conv2D(6, (5, 5), activation='relu', input_shape=(28, 28, 1), padding='same'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(16, (5, 5), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dense(120, activation='relu'))
model.add(layers.Dense(84, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Display model summary
model.summary()
```

$$(5 \times 5 \times 6 + 1) \times 16 = 2416$$

Param #
156
0
2,416
0
0
48,120
10,164
850



2. Ví dụ kiến trúc CNN đơn giản

2. Ví dụ kiến trúc CNN đơn giản

- Mạng LeNet-5



Yann LeCun, Leon Bottou, Patrick Haffner và Yoshua Bengio

PROC. OF THE IEEE, NOVEMBER 1998

1

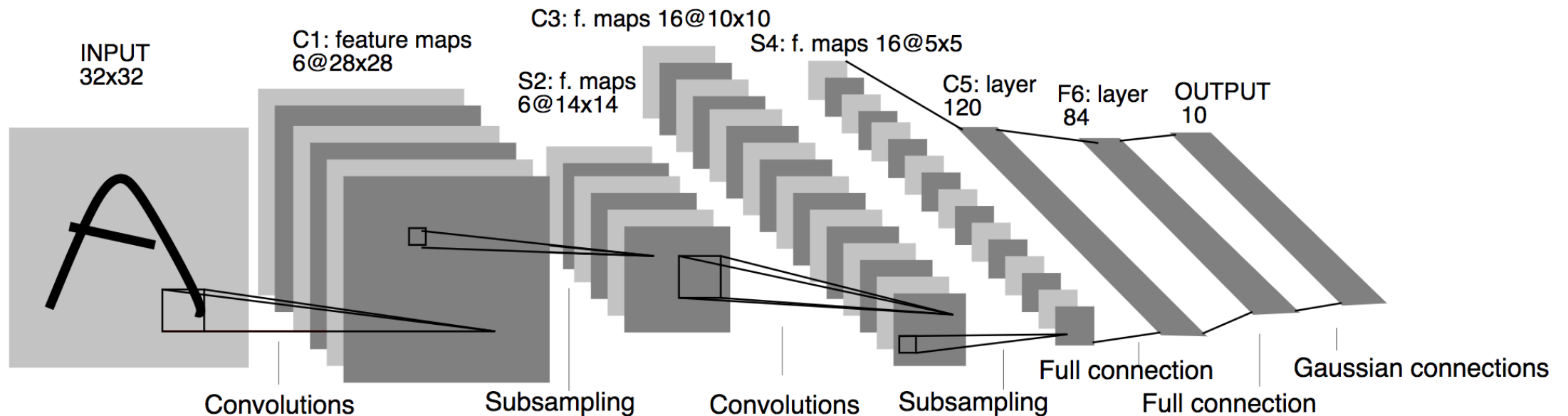
Gradient-Based Learning Applied to Document Recognition

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner

http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf

2. Ví dụ kiến trúc CNN đơn giản

- Mạng LeNet-5



[Tham khảo thêm: \[ML – 21\] Convolution Neural Network : LeNet-5](#)



2. Ví dụ kiến trúc CNN đơn giản

- Mạng LeNet-5

```
# Build LeNet-5 model
model = models.Sequential()
model.add(layers.Conv2D(6, (5, 5), activation='relu', input_shape=(28, 28,
1), padding='same'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(16, (5, 5), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dense(120, activation='relu'))
model.add(layers.Dense(84, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Display model summary
model.summary()
```

Tham khảo thêm: [LeNet-5 — A complete guide](#)



2. Ví dụ kiến trúc CNN đơn giản

- Mạng LeNet-5

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 28, 28, 6)	156
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 6)	0
conv2d_4 (Conv2D)	(None, 10, 10, 16)	2,416
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 16)	0
flatten_1 (Flatten)	(None, 400)	0
dense_3 (Dense)	(None, 120)	48,120
dense_4 (Dense)	(None, 84)	10,164
dense_5 (Dense)	(None, 10)	850

Total params: 61,706 (241.04 KB)

Trainable params: 61,706 (241.04 KB)

Non-trainable params: 0 (0.00 B)

Tham khảo thêm: [LeNet-5 — A complete guide](#)



Huấn luyện và đánh giá mô hình



1. Chuẩn bị dữ liệu

Sử dụng các thư viện liên quan:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report
from tensorflow.keras import datasets, layers, models
from tensorflow.keras.utils import to_categorical
```



1. Chuẩn bị dữ liệu

Chuẩn bị các tập dữ liệu:

```
# Load the MNIST dataset
(train_images, train_labels), (test_images, test_labels) =
datasets.mnist.load_data()

# Split the data into train, validation, and test sets
validation_split = 0.1
validation_size = int(len(train_images) * validation_split)

validation_images = train_images[:validation_size]
validation_labels = train_labels[:validation_size]

train_images = train_images[validation_size:]
train_labels = train_labels[validation_size:]
```




1. Chuẩn bị dữ liệu

Tiền xử lý dữ liệu:

```
# Further preprocessing
train_images = train_images.reshape((len(train_images), 28, 28, 1))
                                     .astype('float32') / 255
validation_images = validation_images.reshape((len(validation_images), 28, 28, 1))
                                                         .astype('float32') / 255
test_images = test_images.reshape((len(test_images), 28, 28, 1))
                                                         .astype('float32') / 255

train_labels = to_categorical(train_labels)
validation_labels = to_categorical(validation_labels)
test_labels = to_categorical(test_labels)
```



2. Cấu hình mô hình huấn luyện

Biên dịch mô hình:

- Optimizer: SGD vs Adam
- Loss function: cross-entropy
- Metrics: accuracy

```
# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

```
# Compile the model
opt = keras.optimizers.Adam(learning_rate=0.01)
model.compile(optimizer= opt, loss='categorical_crossentropy', metrics=['accuracy'])
```



2. Cấu hình mô hình huấn luyện

Huấn luyện mô hình:

- epochs: Số lần lặp
- batch size: số lượng mẫu được đưa vào mô hình trong một lần xử lý.

```
# Train the model with validation data
history = model.fit(train_images, train_labels, epochs=10, batch_size=64,
validation_data=(validation_images, validation_labels))
```



2. Cấu hình mô hình huấn luyện

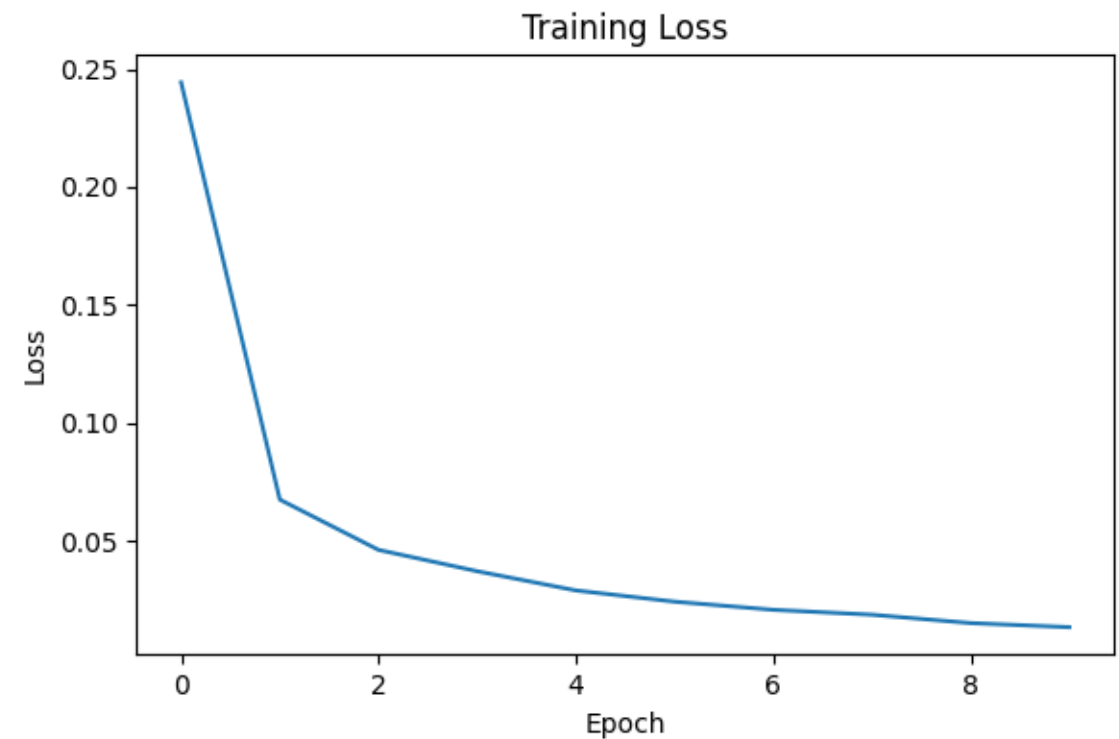
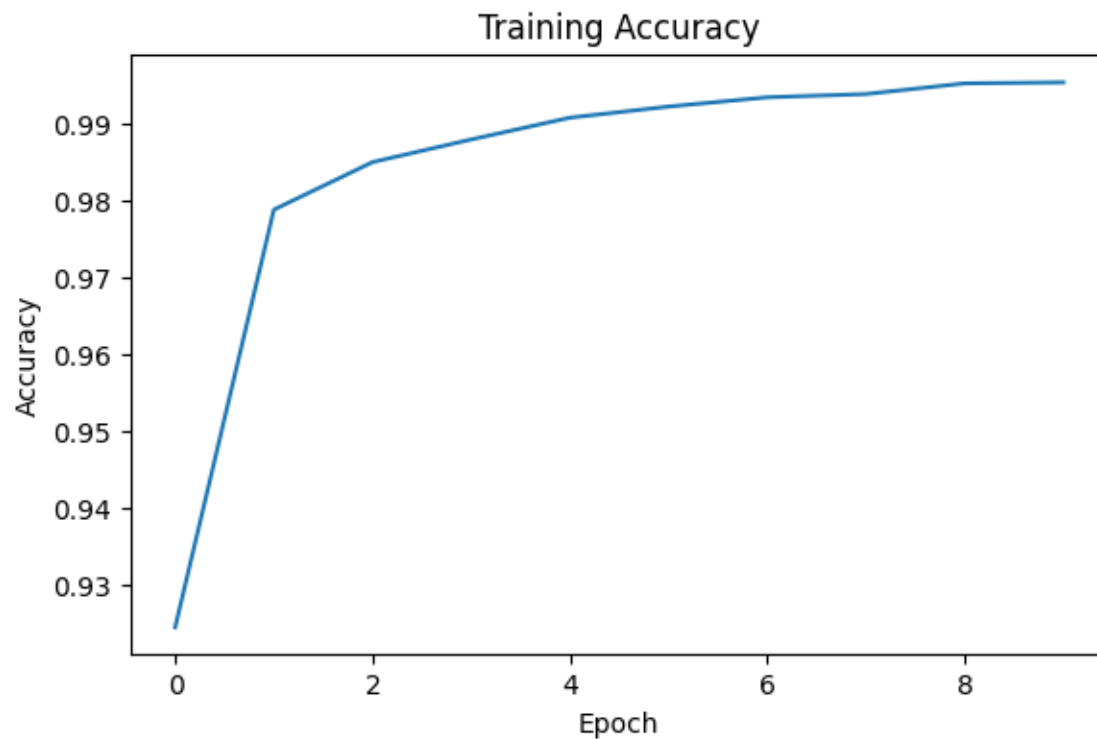
- Quá trình huấn luyện

```
*** Epoch 1/10
844/844 ██████████ 34s 36ms/step - accuracy: 0.8236 - loss: 0.5628 - val_accuracy: 0.9787 - val_loss: 0.0760
Epoch 2/10
844/844 ██████████ 39s 34ms/step - accuracy: 0.9778 - loss: 0.0712 - val_accuracy: 0.9813 - val_loss: 0.0625
Epoch 3/10
844/844 ██████████ 28s 34ms/step - accuracy: 0.9845 - loss: 0.0486 - val_accuracy: 0.9858 - val_loss: 0.0485
Epoch 4/10
844/844 ██████████ 41s 34ms/step - accuracy: 0.9884 - loss: 0.0377 - val_accuracy: 0.9853 - val_loss: 0.0474
Epoch 5/10
844/844 ██████████ 41s 34ms/step - accuracy: 0.9915 - loss: 0.0273 - val_accuracy: 0.9843 - val_loss: 0.0500
Epoch 6/10
844/844 ██████████ 42s 35ms/step - accuracy: 0.9921 - loss: 0.0248 - val_accuracy: 0.9878 - val_loss: 0.0437
Epoch 7/10
844/844 ██████████ 41s 35ms/step - accuracy: 0.9939 - loss: 0.0192 - val_accuracy: 0.9888 - val_loss: 0.0383
Epoch 8/10
317/844 ██████████ 15s 30ms/step - accuracy: 0.9948 - loss: 0.0180
```

Số ảnh của tập train_images: 54000; batch_size = 64 → 54000/64 ~ 844

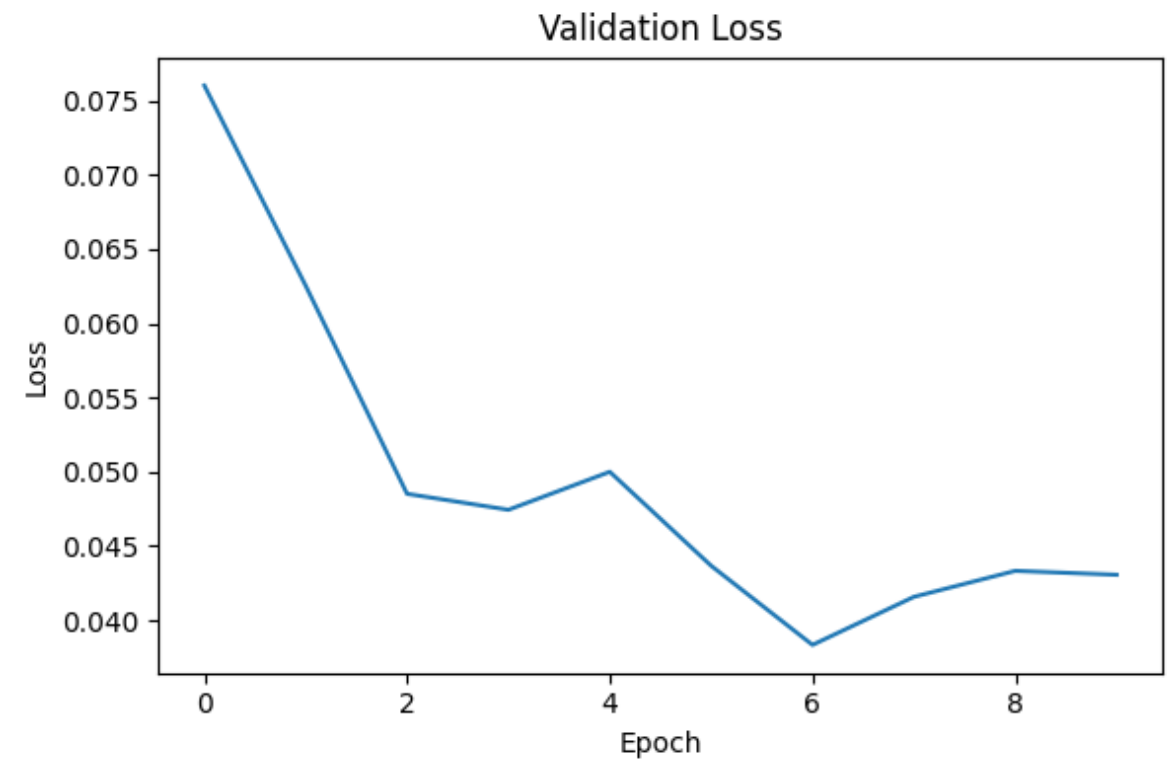
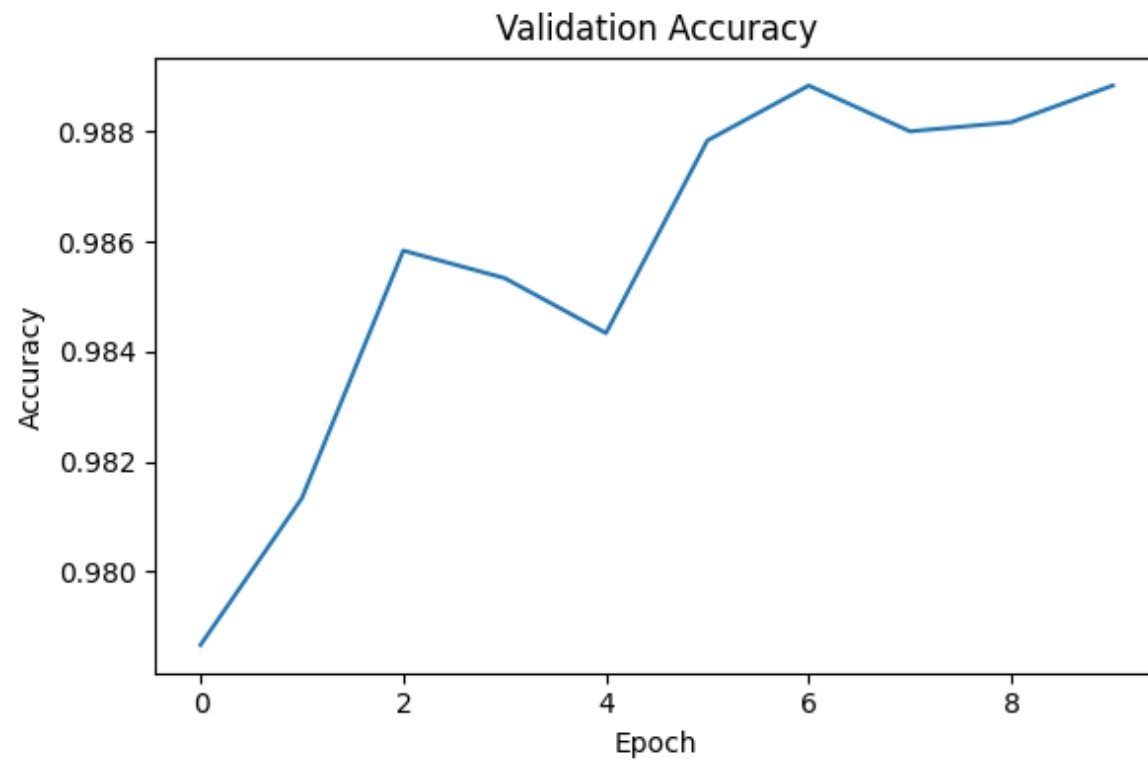
3. Đánh giá mô hình

- Đồ thị loss/accuracy theo epoch



3. Đánh giá mô hình

- Đồ thị loss/accuracy theo epoch



3. Đánh giá mô hình

- Đánh giá loss/accuracy cho tập test

```
# Testing the model on test data
test_loss, test_accuracy = model.evaluate(test_images, test_labels, verbose=2)

print(f"Test Accuracy: {test_accuracy}")
print(f"Test Loss: {test_loss}")
```

```
313/313 - 3s - 10ms/step - accuracy: 0.9918 - loss: 0.0289
Test Accuracy: 0.9918000102043152
Test Loss: 0.028876833617687225
```



3. Đánh giá mô hình

- Đánh giá loss/accuracy cho tập test

```
# Predicting labels for test images
predicted_labels = np.argmax(model.predict(test_images), axis=-1)

# Display classification report
print("Classification Report:\n", classification_report(np.argmax(test_labels,
axis=-1), predicted_labels))
```




3. Đánh giá mô hình

- Đánh giá loss/accuracy cho tập test

```
313/313 ----- 2s 7ms/step
Classification Report:
              precision    recall  f1-score   support

     0           0.99       1.00       0.99         980
     1           0.99       0.99       0.99        1135
     2           0.99       0.99       0.99        1032
     3           0.99       0.99       0.99        1010
     4           0.99       1.00       0.99         982
     5           0.98       0.99       0.99         892
     6           1.00       0.99       0.99         958
     7           1.00       0.99       0.99        1028
     8           0.99       0.99       0.99         974
     9           0.99       0.99       0.99        1009

 accuracy              0.99        10000
 macro avg           0.99       0.99       0.99        10000
 weighted avg        0.99       0.99       0.99        10000
```



Transfer Learning



1. Giới thiệu

- Transfer learning (học chuyển giao) là một kỹ thuật trong Machine Learning: **sử dụng các mô hình đã được đề xuất hoặc đã huấn luyện trước đó** trên một tập dữ liệu để giải quyết các vấn đề khác → **tiết kiệm chi phí** xây dựng mô hình hoặc huấn luyện .
- Có hai phương pháp phổ biến:
 - Tái sử dụng kiến trúc
 - Tái sử dụng các trọng số của mô hình

Transfer Learning

- VGG16



UNIVERSITY OF
OXFORD



VisionFactory
www.visionfactory.co

Very Deep ConvNets for Large-Scale Image Recognition

Karen Simonyan, Andrew Zisserman
Visual Geometry Group, University of Oxford

ILSVRC Workshop
12 September 2014

Published as a conference paper at ICLR 2015

VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

Karen Simonyan* & **Andrew Zisserman***

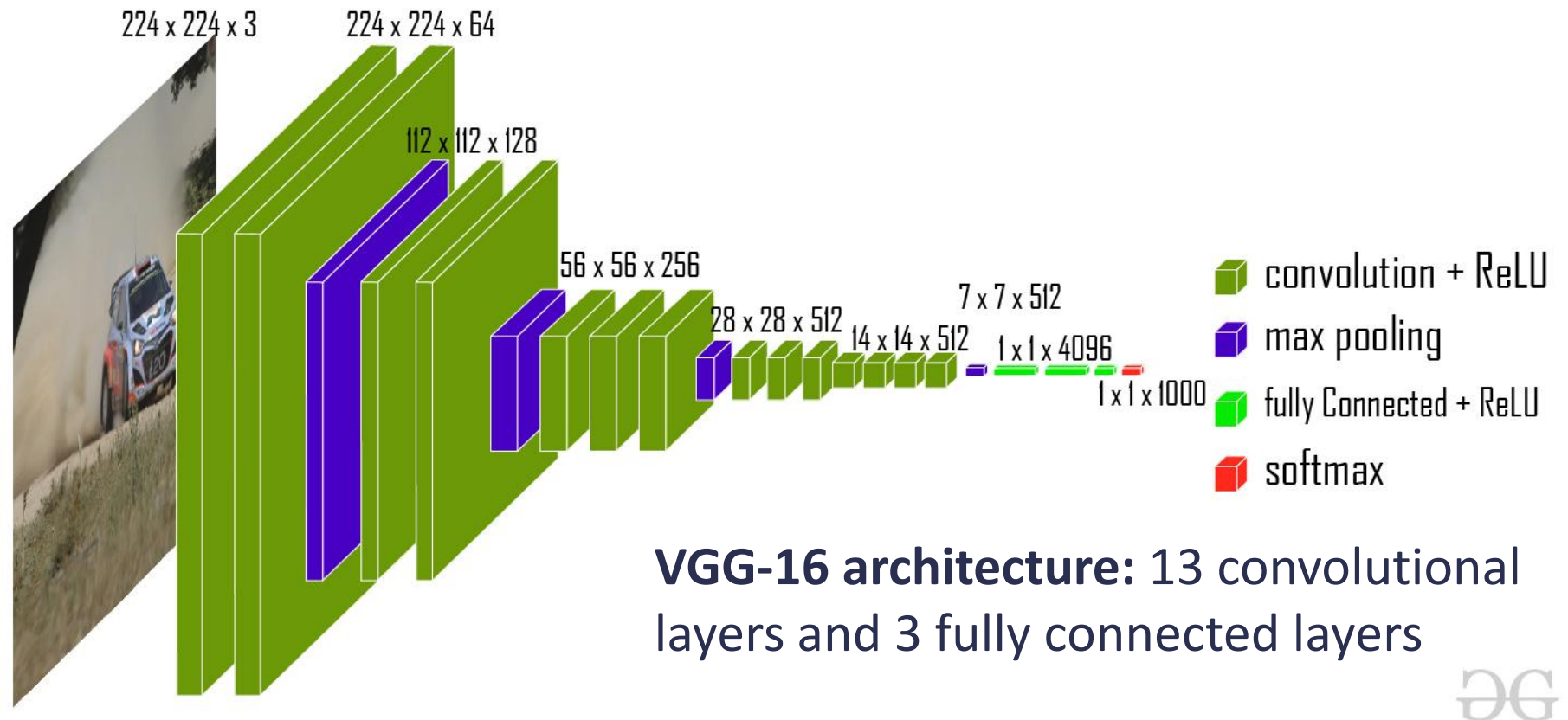
Visual Geometry Group, Department of Engineering Science, University of Oxford
{karen, az}@robots.ox.ac.uk

<https://arxiv.org/pdf/1409.1556v6>

https://www.robots.ox.ac.uk/~karen/pdf/ILSVRC_2014.pdf

Transfer Learning

- **VGG16**



Nguồn: <https://www.geeksforgeeks.org/vgg-16-cnn-model/>



Transfer Learning

- **VGG16**

```
# load model VGG16
based_model = vgg16.VGG16(
    weights = 'imagenet',
    include_top = False,
    input_shape = (img_rows, img_cols, 3)
)

# Freeze layers, not training these layers
for layer in based_model.layers:
    layer.trainable = False
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ————— 0s 0us/step



Transfer Learning

- **VGG16:** Xây dựng mô hình mới cho bài toán mới

```
def layer_added(output_based_network, num_classes):  
    x = output_based_network  
    x = layers.Flatten()(x)  
    x = layers.Dense(1024, activation='relu')(x)  
    x = layers.Dense(256, activation='relu')(x)  
    x = layers.Dense(num_classes, activation='softmax')(x)  
  
    return x
```




Transfer Learning

- **VGG16:** Xây dựng mô hình mới cho bài toán mới

```
output_based_network = based_model.output  
output_layer = layer_added(output_based_network, num_classes)  
model = Model(based_model.input, output_layer)
```




Transfer Learning

- **VGG16:** Thực hiện biên dịch và huấn luyện mô hình

```
# compile model
model.compile(optimizer = tf.keras.optimizers.Adam() ,
              loss = tf.keras.losses.CategoricalCrossentropy() ,
              metrics=['acc'])

# fit model
history = model.fit(X_train, y_train, validation_data=(X_val, y_val) ,
                  batch_size=32,
                  epochs=20)
```



Tài liệu tham khảo

- [CS231n - Convolutional Neural Networks](#)
- <https://cs231n.github.io/convolutional-networks/>
- [PyTorch Tutorials](#)
- [TensorFlow/Keras Tutorials](#)
- <https://stanford.edu/~shervine/l/vi/teaching/cs-230/cheatsheet-convolutional-neural-networks>
- <https://viblo.asia/p/deep-learning-tim-hieu-ve-mang-tich-chap-cnn-maGK73bOKj2>
- <https://www.analyticsvidhya.com/blog/2021/05/convolutional-neural-networks-understand-the-basics/>
- https://d2l.ai/vn.com/chapter_convolutional-neural-networks/lenet_vn.html
- <https://viblo.asia/p/mang-no-ron-tich-chap-p1-DZrGNNjPGVB>
- [Python Tensorflow – tf.keras.layers.Conv2D\(\) Function | GeeksforGeeks](#)
- <https://dlapplications.github.io/2018-07-06-CNN/>
- <https://poloclub.github.io/cnn-explainer/>